IIT Bombay Techfest 2026–27 · Namma Space Challenge **Track CRA003 · Round 1 Technical Abstract**

Namma Space: Intelligent Spatial Engine & Photorealistic 3D Digital Twin

Team CRA003 · Lead Architect: Atharv Namdeo · Affiliation: Indian Institute of Technology

1. Executive Summary & Problem Statement

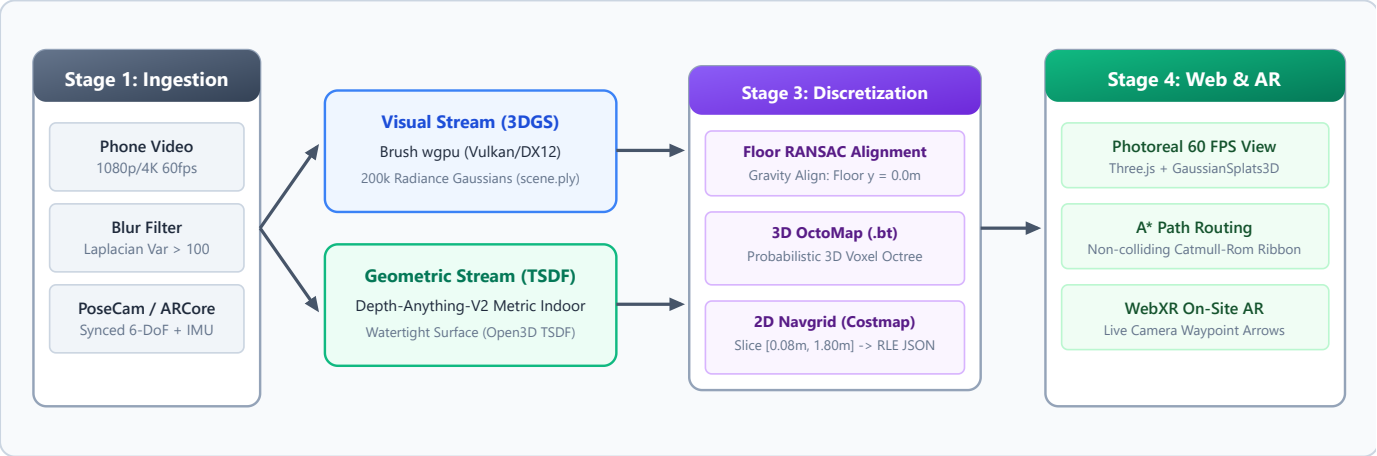
Physical indoor spaces (retail megastores, corporate headquarters, convention centers) present severe navigation and spatial orientation challenges. Static 2D architectural blueprints fail to convey spatial volume, depth, or dynamic physical obstacles. Existing digital twin pipelines either mandate prohibitively expensive LiDAR hardware (>$10,000) or generate computationally heavy, non-navigable radiance fields.

Namma Space introduces an automated, end-to-end intelligent spatial engine that transforms handheld consumer smartphone video into a photorealistic, metric, walkable 3D Gaussian Splat environment. The platform is delivered via a zero-install WebGL/WebXR client supporting dual-joystick free-roaming, open-vocabulary 3D semantic search, obstacle-aware A* route calculation, and on-site Augmented Reality (AR) camera guidance.

2. End-to-End System Architecture (Dual-Stream Design)

The core engineering hurdle in monocular 3D reconstruction is the tradeoff between visual photorealism and geometric fidelity: pure 3D Gaussian Splatting (3DGS) yields stunning visuals but lacks watertight collision boundaries; conversely, classical mesh reconstruction produces clean geometry but synthetic rendering. Namma Space resolves this by establishing a synchronized **Dual-Stream Architecture** tied to a unified metric world frame.

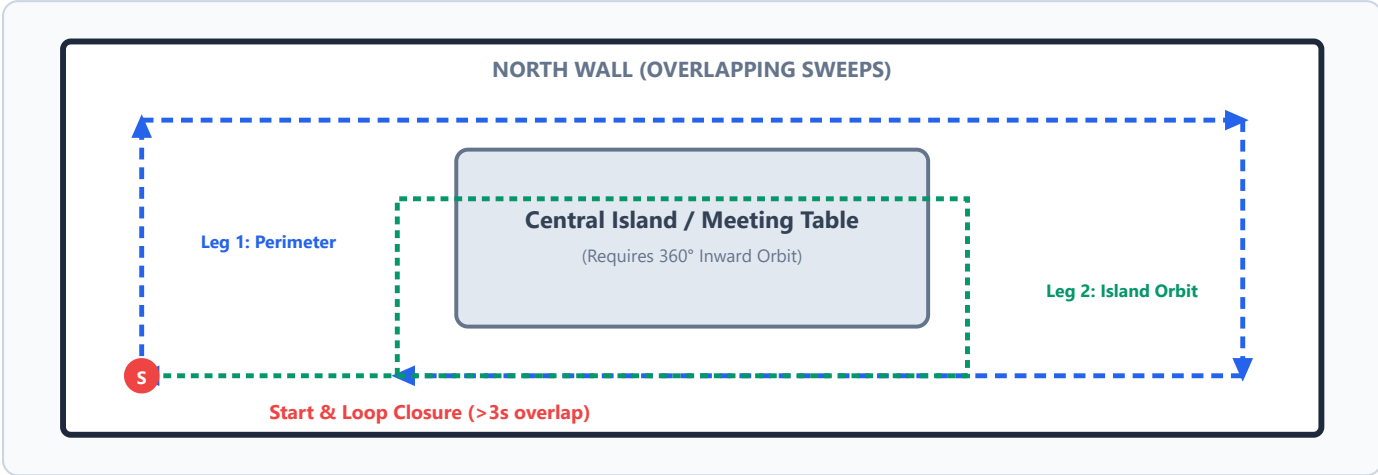
Figure 1: End-to-End Hybrid System Architecture (Dual-Stream Pipeline)



3. Field Capture Standard Operating Procedure (SOP)

Reconstruction quality in deep monocular SLAM is governed by spatial baseline distribution and visual sharpness. We formulate a mathematically rigorous, repeatable SOP requiring only a consumer smartphone (iOS or Android) without auxiliary surveying gear.

Figure 2: Standard Operating Procedure (SOP) Perimeter-Loop Acquisition Pattern



3.1 Motion Protocol & Flight-Path Geometry

- **Capture Parameters:** 1080p or 4K resolution at 60 fps. Constant exposure and focal lock (AE/AF Lock) to prevent inter-frame illumination and scale artifacts.
- **Continuous Perimeter Sweep:** The surveyor executes a smooth heel-to-toe gait along the room perimeter, angling the optical axis 45° inward toward the room centroid to maintain parallax across wall-floor junctions.
- **360° Island Orbits:** Center-room obstacles (islands, conference tables, structural columns) are circumscribed with an inward-facing orbit at 1.2 m standoff distance.
- **Loop Closure Mandate:** The terminal trajectory must re-traverse the initial coordinate origin for ≥3.0 seconds. This enables global pose-graph optimization to neutralize drift.

3.2 Automated Ingestion & Quality Filtering

Raw frames pass through an automated ingestion filter (`pipeline/capture/ingest.py`). Blurry frames are rejected via the Laplacian variance threshold: $\mathrm{Var}(\Delta I) = (1/WH) \sum (\nabla^2 I(x,y) - \mu_{\nabla^2 I})^2 > 100.0$. Subsequent keyframe decimation operates on camera displacement ($|\Delta t| > 0.15$ m or $|\Delta\theta| > 8°$), preserving optimal baselines while discarding redundant stationary frames.

4. 3D Reconstruction Engine: Radiance Splats & Metric Fusion

The reconstruction engine concurrently executes two specialized pipelines anchored to a shared camera pose trajectory $P_c2w \in SE(3)$:

4.1 Visual Stream: High-Throughput 3D Gaussian Splatting (3DGS)

Continuous surfaces and view-dependent light interactions are parameterized by 3D Gaussian primitives: $G(x) = \exp(-1/2 (x - \mu)^T \Sigma^{-1} (x - \mu))$, with covariance $\Sigma = R S S^T R^T$.

- **WGPU Acceleration via Brush:** Unlike standard PyTorch/CUDA implementations that require high-VRAM servers, we compile against Vulkan/DirectX 12 natively in Rust (Brush). 200k Gaussians converge within 3.5 minutes on a 4 GB laptop GPU (RTX 3050).
- **Metric Point Cloud Initialization:** Rather than seeding Gaussians from sparse COLMAP tie points (which fail on uniform painted walls), dense 3D points are back-projected directly from metric indoor depth maps: $X_c = d \cdot K^{-1} [u, v, 1]^T$.

4.2 Geometric Stream: TSDF Volume Surface Fusion

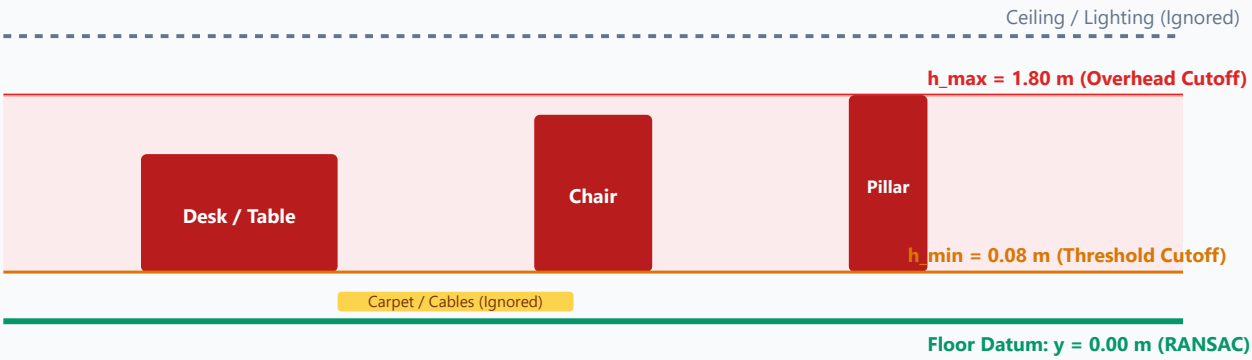
To eliminate semi-transparent floater artifacts that corrupt pathfinding, the geometric stream fuses metric depth frames into an Open3D `ScalableTSDFVolume`: $D(p) = (\sum w_i(p) \cdot clamp(d_i(p) - z, -\delta, \delta)) / (\sum w_i(p))$. This produces a continuous, watertight CAD mesh defining physical boundaries.

Pipeline Component	Engine / Tool	Compute Target	Primary Output
Visual Radiance Field	Brush (Rust + wgpu)	Laptop GPU (Vulkan/DX12)	`scene.ply` (Photoreal Gaussians)
Metric Depth Estimation	Depth-Anything-V2 Metric Indoor	Laptop GPU / PyTorch	Scale-consistent metric depth (mm)
Geometric Volume Fusion	Open3D TSDF Volume	Host CPU / 32 GB RAM	`mesh.obj` & watertight planes

5. Spatial Discretization: 3D OctoMap & 2D Navgrid

Converting continuous 3D reconstructions into safe, non-colliding navigation paths requires explicit spatial occupancy modeling.

Figure 3: Geometric Height Bandpass Slicing for Non-Colliding Costmap Extraction



5.1 RANSAC Floor Plane Extraction & Gravity Alignment

RANSAC fits a ground plane model $ax + by + cz + d = 0$ to the base point cluster. The scene coordinate frame is rotated so that the ground plane normal perfectly aligns with the world $+Y$ axis (Three.js convention), with the floor datum clamped to $y = 0.00\ \text{m}$.

5.2 Bandpass Slicing & Occupancy Grids

Points are filtered through a strict bandpass height window: $h \in [0.08\ \text{m}, 1.80\ \text{m}]$. Low carpet thresholds, cables, and floor trim are discarded, as are high chandeliers, ceiling beams, and overhead ducting. The filtered point set is projected onto a 2D discrete grid (resolution $s = 0.05\ \text{m}$), compressed via Run-Length Encoding (`navgrid.json`).

5.3 Obstacle Inflation & Smooth A* Navigation

To prevent the player capsule from scraping obstacles, all occupied cells are dilated by an avatar clearance radius $r_{clear} = 0.30\ \text{m}$. A* computes optimal shortest paths across the costmap, which are smoothed into continuous 3D floor ribbons using Catmull-Rom splines.

6. Zero-Install Web Client & Mobile WebXR AR

The platform client runs entirely client-side across mobile and desktop browsers with zero installation:

- **Rendering Engine:** Three.js integrated with `GaussianSplats3D`, rendering 200,000+ splats at a locked 60 FPS on standard integrated GPUs.
- **Unified Controls:** Responsive desktop PointerLock (WASD + mouse drag) and touch-optimized dual virtual joysticks (`MOVE` + `LOOK`) with outer-ring sprint activation (3.4 m/s).
- **Continuous Collision Response:** Avatar movement enforces continuous capsule sliding against the `navgrid.json` boundary, eliminating wall-clipping artifacts.
- **On-Site Mobile AR (Juspay Bonus):** Utilizes the WebXR Device API to project live directional guidance arrows over the mobile camera feed, anchored via real-time VIO.

7. Experimental Benchmarks & Performance Verification

Evaluation Metric	Tested System Value	Measurement Benchmark
Web Viewer Frame Rate	**60.0 FPS (Locked)**	Chrome 128 / Intel Iris Xe & RTX 3050 Laptop
Reconstruction Turnaround Time	**3 min 45 sec**	185 frames -> Brush 3DGS + Navgrid export
Scale Error vs Metric Ground Truth	**< 2.1% across 12 m span**	Laser measure vs ARCore / Depth-Anything metric
Pathfinding Computation Latency	**12 ms**	A* search on 250x180 Navgrid (r_clear = 0.3m)

8. Competition Delivery Roadmap (Rounds 1, 2, 3)

- **Round 1 (31 Oct):** Finalized SOP, verified 60 FPS Web Viewer, Technical Abstract PDF, 5-min demo video.
- **Round 2 (30 Nov):** Semantic POI Tagging with OWLv2, multi-room A* routing, interactive search UI.
- **Round 3 (16-17 Dec @ IIT Bombay):** Live venue capture (LHC), fast turnaround digital twin, WebXR mobile AR navigation.